Numerical Optimization

Qingyuan Fang [1]

[1] Johns Hopkins University

February 23, 2026

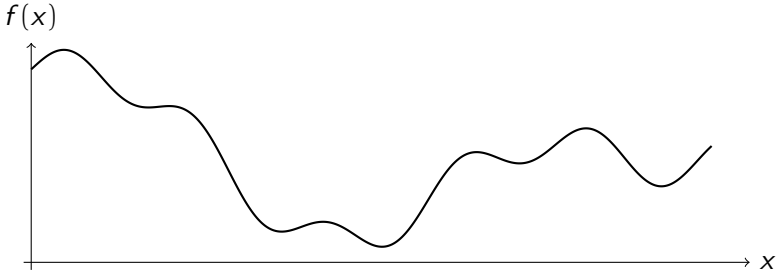
Roadmap

- **Part I** Simulated Annealing (SA)
- **Part II** Stochastic Gradient Descent (SGD)

Simulated Annealing

- Inspired by the physical process of heating and slow cooling (annealing).

Exploration (high temperature) **v.s.** **Exploitation (low temperature)**



- **Key idea**: occasionally accept worse moves early to escape local minima
- Pro: global optimizer, non-convex/discontinuous objectives, large search spaces
- Con: parameter tuning matters, can be slow, no guarantee

Procedure

State $x \in \mathcal{X} \subseteq \mathbb{R}^d$. Start from initial guess x_0 and initial temperature T_0. Iterate:

1. Propose a neighbor $x' \sim q(\cdot \mid x)$.
2. Accept ($x_{new} = x'$) or reject ($x_{new} = x$) based on a rule that depends on the temperature.
3. Gradually lower temperature T.

Insight: (Under certain conditions) At fixed T, the simulated draws converge to the *Gibbs distribution*

$$p(x) = \frac{\exp\left[-\frac{f(x)}{T}\right]}{\int_{\tilde{x} \in \mathcal{X}} \exp\left[-\frac{f(\tilde{x})}{T}\right]}$$

Scaling and parameter transforms

Good practice before SA:

- Standardize parameters to similar units.
- Enforce positivity via logs: $\theta > 0 \Rightarrow \phi = \log \theta$.
- Map to $[0, 1]$ via logit: $p = \frac{1}{1+\exp(-\phi)}$.

(a) Proposal design

Typical continuous-domain choice:

$$x' = x + s_t \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I_d).$$

Design choices:

- **Step scale** s_t: too small $\Rightarrow$ slow mixing; too large $\Rightarrow$ low acceptance
- **Constraints**: reflect/clip/penalize at bounds

Constraints and bounds

For box constraints $l \leqslant x \leqslant u$:

- **Reflect:** if $x_i' < l_i$, set $x_i' = 2l_i - x_i'$ (similarly for u_i).
- **Project:** clip $x_i' \leftarrow \min(\max(x_i', l_i), u_i)$.
- **Penalize:** optimize $f(x) + \lambda \cdot \text{violation}(x)$.

In econometrics: parameter transforms often give cleaner geometry (next slide).

(b) Metropolis algorithm

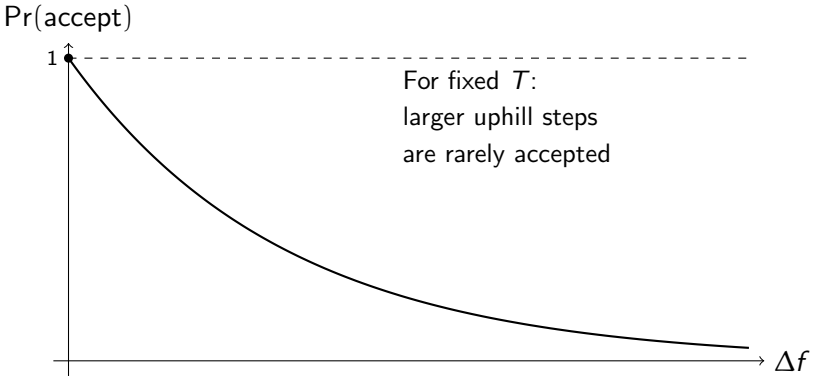
Let $\Delta f = f(x') - f(x)$.

$$\Pr(\text{accept } x') = \begin{cases} 1, & \Delta f \leqslant 0, \\ \exp(-\Delta f / T), & \Delta f > 0. \end{cases}$$

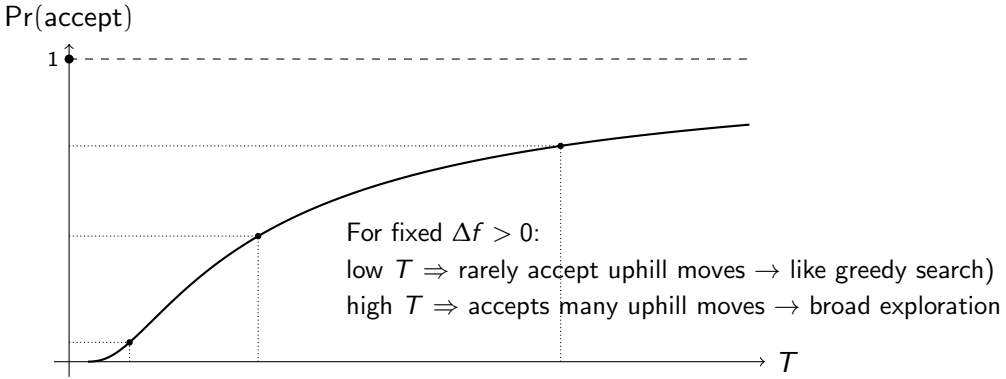
Interpretation:

- Always accept improvements.
- Sometimes accept worse moves; probability decreases with Δf and as $T \downarrow$.

Acceptance probability



Temperature = exploration vs exploitation knob



- SA performance is determined by:

  (i) proposal scale s_t, (ii) cooling schedule T_t, (iii) time spent per temperature.

(c) Cooling schedules

We choose a sequence $T_0 \geqslant T_1 \geqslant \ldots \downarrow 0$.

- Exponential: $T_{k+1} = \alpha T_k$, $\alpha \in (0, 1)$ (workhorse in practice).

Tradeoff: slower cooling (bigger α) is safer but more expensive.

Rule of thumb

- Choose T_0 so that early acceptance rate is high (e.g., 60–90%).
- For each temperature level, run m proposals.
- Cool: $T \leftarrow \alpha T$ with $\alpha \approx 0.90$ to 0.99.

Stopping criteria

Common stopping conditions (may combine several):

- Temperature below threshold: $T < T_{\min}$.
- Budget exhausted: max iterations / max function evaluations.
- Stall: best-so-far improves by less than ε over K steps.
- Acceptance rate too low for too long.

Remember: SA is stochastic $\Rightarrow$ validate with multiple runs.

Example 1: Rosenbrock (non-convex valley)

Wikipedia.

$$f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2.$$

Features:

- Global minimum at $(1, 1)$ with $f = 0$.
- Long narrow curved valley $\Rightarrow$ difficult to converge to the global minimum.

Goal: Useful diagnostics, robust optimization routine

Example 2: Periodic nonlinear least squares

Toy model with many local minima in frequency:

$$y_t = \alpha + \beta \sin(\omega t) + \varepsilon_t, \quad t = 1, \ldots, T.$$

Estimate $\theta = (\alpha, \beta, \omega)$ by minimizing SSE:

$$\min_{\theta} \sum_{t=1}^{T} \left(y_t - \alpha - \beta \sin(\omega t)\right)^2.$$

The SSE surface is often multi-modal in ω (especially with short samples).

Goal: Hybrid workflow (SA + local search).

Takeaways: SA

- SA = Metropolis acceptance + cooling; robust to rugged objectives.
- Tuning matters: scaling, T_0, step sizes, cooling rate, restarts, etc.
- Best practice: SA for global search + local optimizer for polishing.

Further reading:

Kirkpatrick, Gelatt, Vecchi (1983), *Science*.

Bertsimas & Tsitsiklis (1993), *Statistical Science*.

Sen & Stoffa (2013), "Simulated annealing methods," in *Global Optimization Methods in Geophysical Inversion*, Cambridge University Press, pp. 81–118.

Other global optimizers

See:

- TikTak
- NLopt Algorithms

Reference:

Arnoud, Guvenen & Kleineberg (2019), "Benchmarking Global Optimizers," *NBER Working Paper* No. 26340.

Setup and notation

Parameter and loss:

- Parameter vector: $\theta \in \mathbb{R}^d$.
- Per-observation loss: $\ell(\theta; z)$, where z is one data point.
- Dataset: $\{z_i\}_{i=1}^n$.

Goal:

$$\min_{\theta \in \mathbb{R}^d} f(\theta) \equiv \frac{1}{n} \sum_{i=1}^{n} \ell(\theta; z_i).$$

Gradient notation:

- Full gradient $[d \times 1]$: $\nabla f(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell(\theta; z_i)$

Full-batch gradient descent

Gradient descent (GD) uses the exact gradient of the full objective:

$$\theta_{t+1} = \theta_t - \eta_t \nabla f(\theta_t), \qquad t = 0, 1, 2, \ldots$$

where $\eta_t > 0$ is the learning rate (step size).

Pros

- Deterministic update.
- Stable descent direction.

Cons

- Each step costs $O(n)$ gradient evaluations $\Rightarrow$ Expensive for large n.

Mini-batch SGD

Idea of SGD: replace $\nabla f(\theta_t)$ by a cheap random estimator g_t with

$$\mathbb{E}[g_t \mid \theta_t] = \nabla f(\theta_t) \quad \text{(unbiasedness)}.$$

Sampling: Choose i.i.d. indices $I_{t,1}, \ldots, I_{t,b} \sim \mathrm{Unif}\{1, \ldots, n\}$, and define

$$g_t = \frac{1}{b} \sum_{j=1}^{b} \nabla \ell(\theta_t; z_{I_{t,j}}).$$

Update:

$$\theta_{t+1} = \theta_t - \eta_t g_t.$$

Key tradeoff

- Cheaper per step than GD ($b << N$).
- But noisier direction, so the iterates may fluctuate.

Momentum

Plain SGD can oscillate when:

- gradients are noisy (batch size too small), or
- curvature differs strongly across directions (ill-conditioning).

Momentum idea Why Momentum Really Works?

- Average gradients over time to reduce high-frequency noise.
- Build "inertia" in persistent descent directions.

$$g_t = \frac{1}{b}\sum_{j=1}^{b} \nabla \ell(\theta_t; z_{I_{t,j}}),$$
$$p_{t+1} = \beta p_t + (1-\beta) g_t,$$
$$\theta_{t+1} = \theta_t - \eta_t p_{t+1},$$

, where $\beta \in [0, 1)$, typically large.

Adam

Adam (**A**daptive **M**oment) combines:

- **Momentum** (first moment / gradient mean),
- **Adaptive scaling** (second moment / squared gradient mean).

Motivation

- Different coordinates of θ may have very different gradient magnitudes.
- Adam rescales each coordinate by an estimate of its recent variability.

High-level idea

$$\text{step in coordinate } j \;\propto\; \frac{\text{smoothed gradient in } j}{\sqrt{\text{smoothed squared gradient in } j} + \varepsilon}.$$

So coordinates with persistently large gradients get smaller effective steps, and vice versa.

Adam

$$g_t = \frac{1}{b}\sum_{j=1}^{b} \nabla \ell(\theta_t; z_{I_{t,j}}).$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t,$$

$$s_t = \beta_2 s_{t-1} + (1-\beta_2)(g_t \odot g_t),$$

where $\beta_1, \beta_2 \in [0, 1)$, typically $\beta_1 = 0.9$, $\beta_2 = 0.999$.

Bias correction (important in early iterations):

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{s}_t = \frac{s_t}{1-\beta_2^t}.$$

Parameter update (elementwise division and square root):

$$\theta_{t+1} = \theta_t - \alpha_t \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \varepsilon}, \text{typically } \varepsilon = 10^{-8}$$